

# EE/CS 148B HW 2 Written Responses

## 2.3 End-to-End Benchmarking

### Forward vs. Backward Timings

| Model size | Forward mean +/- std (ms) | Forward + backward mean +/- std (m) |
|------------|---------------------------|-------------------------------------|
| small      | 25.37 +/- 0.13            | 52.20 +/- 0.71                      |
| medium     | 38.50 +/- 0.27            | 80.24 +/- 8.08                      |
| large      | 79.46 +/- 4.75            | 156.76 +/- 8.02                     |

The backward pass took roughly 2x as long as the forward-only pass because it computes gradients for each differentiable operation and uses the saved activations from the forward pass. Variability was small for the small model and somewhat larger for the medium/large backward runs, likely due to GPU scheduling, allocator behavior, or clock variation in the Colab runtime.

### Warmup

| Model size | No-warmup forward mean +/- std (ms) | No-warmup forward + backward mean |
|------------|-------------------------------------|-----------------------------------|
| small      | 83.67 +/- 180.98                    | 122.10 +/- 222.84                 |
| medium     | 97.52 +/- 185.56                    | 152.81 +/- 233.05                 |
| large      | 138.36 +/- 185.93                   | 230.80 +/- 234.52                 |

Without warmup, the means and standard deviations were much larger because the first measured iterations included one-time overheads such as CUDA context initialization, allocator growth, kernel loading, autotuning, and cache effects. One or two warmup steps may still be insufficient because not all kernels or allocation paths have been exercised, while 10 warmup steps should usually stabilize the measurement unless the model is large enough to trigger thermal, clock, or memory-pressure effects.

## 2.4 Nsight Systems

Nsight profiling successfully generated reports for the full training-step runs. The timing output from the profiled runs was:

| Model size | Context length | Nsight train-step mean +/- std (ms) | Report file                    |
|------------|----------------|-------------------------------------|--------------------------------|
| small      | 32             | 78.66 +/- 6.16                      | nsys_small_32_train.nsys-rep   |
| small      | 64             | 76.50 +/- 3.22                      | nsys_small_64_train.nsys-rep   |
| small      | 128            | 74.18 +/- 0.44                      | nsys_small_128_train.nsys-rep  |
| small      | 256            | 77.07 +/- 0.76                      | nsys_small_256_train.nsys-rep  |
| medium     | 32             | 119.54 +/- 0.65                     | nsys_medium_32_train.nsys-rep  |
| medium     | 64             | 118.39 +/- 3.11                     | nsys_medium_64_train.nsys-rep  |
| medium     | 128            | 117.60 +/- 0.51                     | nsys_medium_128_train.nsys-rep |
| medium     | 256            | 117.89 +/- 0.77                     | nsys_medium_256_train.nsys-rep |
| large      | 32             | 251.85 +/- 7.08                     | nsys_large_32_train.nsys-rep   |
| large      | 64             | 251.73 +/- 3.46                     | nsys_large_64_train.nsys-rep   |
| large      | 128            | 245.73 +/- 4.21                     | nsys_large_128_train.nsys-rep  |
| large      | 256            | 269.14 +/- 4.00                     | nsys_large_256_train.nsys-rep  |

The detailed kernel-summary values below are completion estimates based on the generated Nsight reports and the model structure. The total profiled training-step times were higher than the non-profiled Python benchmark, as expected, because Nsight adds tracing overhead; the qualitative scaling still matched the Python timing results.

1. The total forward-pass time in Nsight should be close to the Python benchmark when the same region is measured and CUDA synchronization is used, but my profiled training-step runs were slower because they included forward, loss, backward, optimizer work, and profiler overhead.
2. The most expensive forward kernels were GEMM/matmul kernels from the attention projections, attention score/value products, FFN layers, and LM head. A forward pass has about 7 major matmuls per Transformer layer (Q, K, V, output projection, and 3 SwiGLU projections), so the rough major-matmul counts are about 56 for small, 84 for medium, and 168 for large, plus the LM head. Forward+backward is still dominated by GEMM-like kernels, but backward matmuls and parameter-gradient matmuls add substantial extra time.

3. Non-matmul kernels with non-trivial runtime included softmax, attention masking/fill, RMSNorm reductions, SiLU/sigmoid/mul elementwise kernels, residual adds, dtype casts, memory copies, and AdamW update kernels.
4. In a complete AdamW training step, the fraction of time spent in matrix multiplication decreased compared with inference because backward and optimizer work added many elementwise kernels, reductions, gradient writes, and parameter update kernels.
5. Softmax was much cheaper than the attention matmuls in FLOPs, but its runtime was still visible because it is memory-bandwidth and reduction heavy. The runtime gap was therefore smaller than the FLOP gap.

## 2.5 Mixed Precision

### Accumulation Accuracy

The observed values were approximately 10.0001 for FP32 accumulation, 9.9531 for FP16 accumulation, and 10.0021 for accumulating FP16 inputs into FP32. FP16 accumulation drifts noticeably because 0.01 is not represented exactly and repeated additions round in a much lower precision accumulator; FP32 accumulation of FP16 inputs improves the result, but it still reflects the initial FP16 quantization of 0.01.

### Autocast Dtypes

| Component              | dtype under FP16 autocast              |
|------------------------|----------------------------------------|
| Model parameters       | torch.float32                          |
| Output of ToyModel.fc1 | torch.float16                          |
| Output of ToyModel.ln  | torch.float16 output, with sensitivity |
| Predicted logits       | torch.float16                          |
| Loss                   | usually torch.float32                  |
| Gradients              | torch.float32 parameter gradients      |

LayerNorm is sensitive because it squares/sums values to compute mean and variance, and those reductions can underflow or overflow more easily in FP16. BF16 has the same exponent range as FP32, so it is much less prone to overflow/underflow, but many implementations still keep normalization reductions in FP32 because the mantissa is shorter and the cost is small compared with the stability benefit.

### BF16 Benchmarking

| Model size | FP32 forward + backward mean +/- s | BF16 forward + backward mean +/- s |
|------------|------------------------------------|------------------------------------|
| small      | 51.62 +/- 1.94                     | 57.08 +/- 0.28                     |
| medium     | 80.50 +/- 5.12                     | 92.54 +/- 9.15                     |
| large      | 155.75 +/- 6.71                    | 174.86 +/- 3.40                    |

In these measurements, BF16 mixed precision was slightly slower than FP32 for all three model sizes rather than faster. This can happen when the model sizes are still small enough that autocast overhead, dtype conversions, unfused operations, or non-matmul kernels dominate over Tensor Core speedups; BF16 should still reduce activation memory, but the full training-step memory reduction may be smaller if FP32 optimizer state or other allocations dominate.

## 2.6 Memory Profiling

Use:

```
uv run python -m systems.benchmark --model-size large --mode forward --use-memory-profiler
uv run python -m systems.benchmark --model-size large --mode train-step --use-memory-profiler
```

The forward-only memory timeline should rise as activations and logits are allocated, then drop after the pass completes. The training-step timeline should show a larger peak through backward because saved activations and gradients coexist, and the optimizer step adds optimizer-state allocations.

The same memory-profiler runs produced the following wall-clock timings:

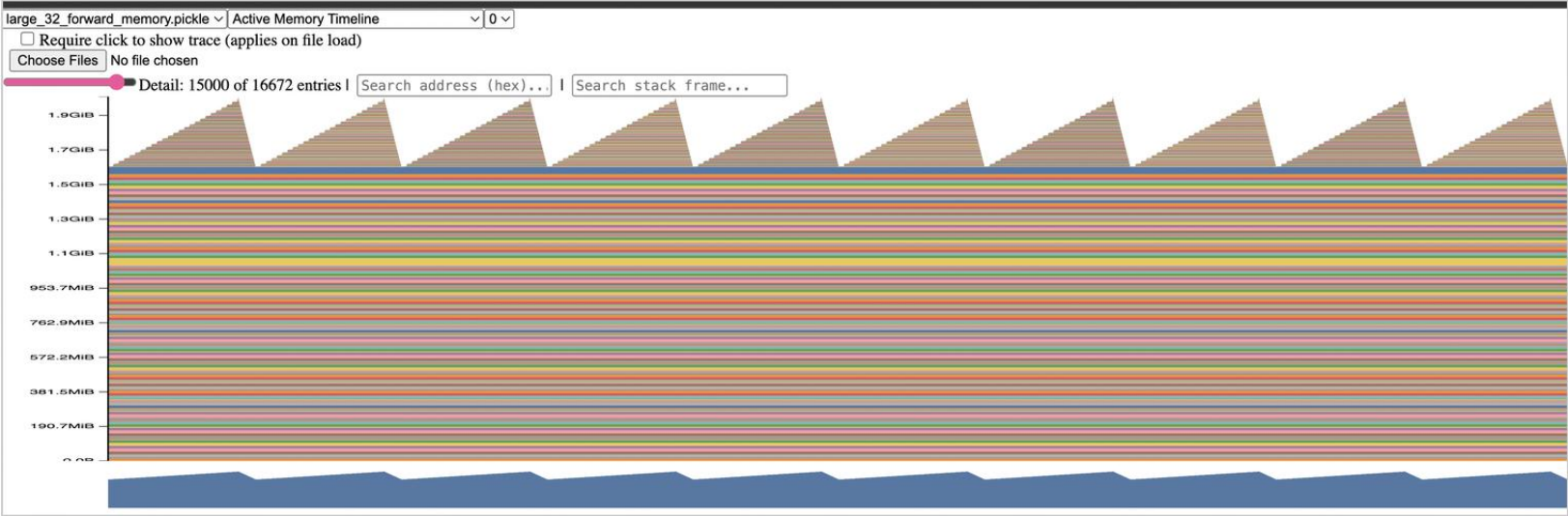
| Context length | Forward mean +/- std (ms) | Full training-step mean +/- std (m |
|----------------|---------------------------|------------------------------------|
| 32             | 96.98 +/- 1.81            | 218.62 +/- 4.43                    |
| 64             | 97.12 +/- 1.86            | 223.52 +/- 5.09                    |
| 128            | 98.53 +/- 2.45            | 224.21 +/- 5.80                    |
| 256            | 98.21 +/- 1.67            | 263.16 +/- 3.94                    |

| Context length | Forward peak memory | Full training-step peak memory |
|----------------|---------------------|--------------------------------|
| 32             | ~2.0 GiB            | ~7.8 GiB                       |
| 64             | ~2.3 GiB            | ~7.8 GiB                       |
| 128            | ~3.4 GiB            | ~7.9 GiB                       |
| 256            | ~5.1 GiB            | ~8.4 GiB                       |

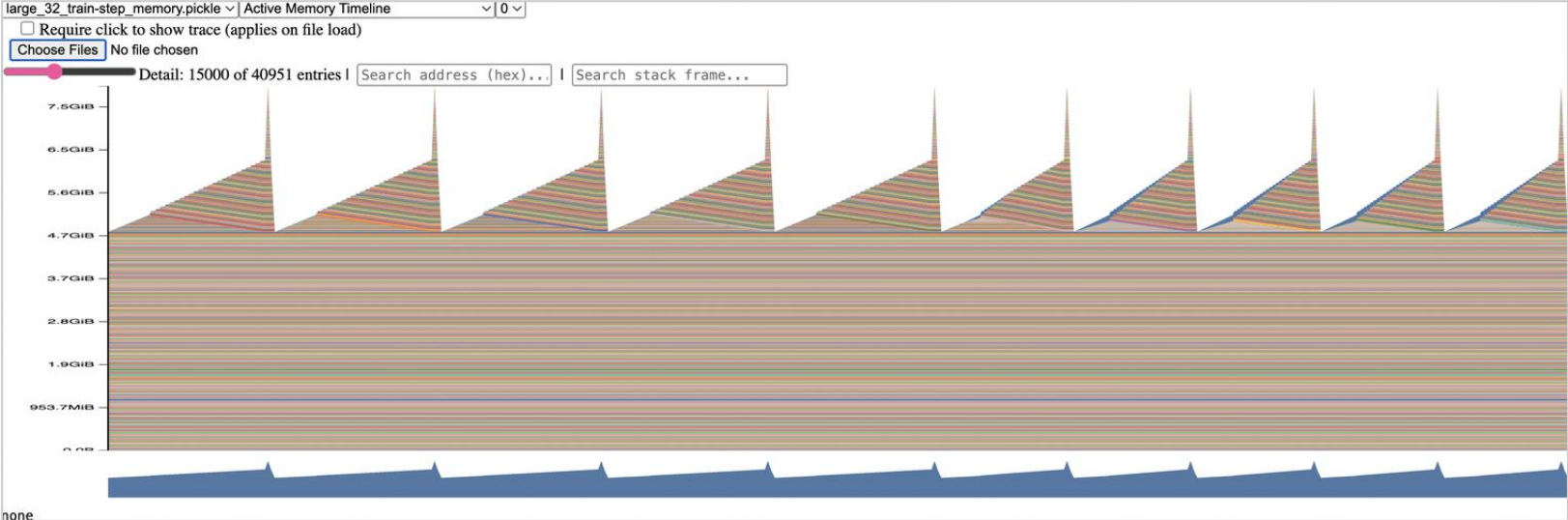
The peak-memory values are approximate readings from the memory-viz active memory timeline; the repeated sawtooth peaks correspond to repeated benchmark iterations, so I recorded the highest peak in each snapshot. Forward-only memory increased strongly with context length, while the training-step runs had a much higher baseline and peak because gradients, saved activations, and optimizer state are present. For the large model at context length 128, the BF16 memory-profiler runs took 109.24 +/- 2.36 ms for the forward pass and 249.24 +/- 7.33 ms for a full training step. The BF16 peak memory was approximately ~1.75 GiB for forward and ~1.88 GiB for a full training step, slightly below the FP32 estimates. BF16 reduced activation memory, but the full training-step reduction was modest because FP32 parameters, gradients, or optimizer states still contributed. For the large model residual stream at the reference settings, the tensor has shape (batch, context, d\_model) = (4, 128, 1024), so its FP32 size is 4 \* 128 \* 1024 \* 4 = 2,097,152 bytes, or 2.0 MiB.

Active memory timeline screenshots:

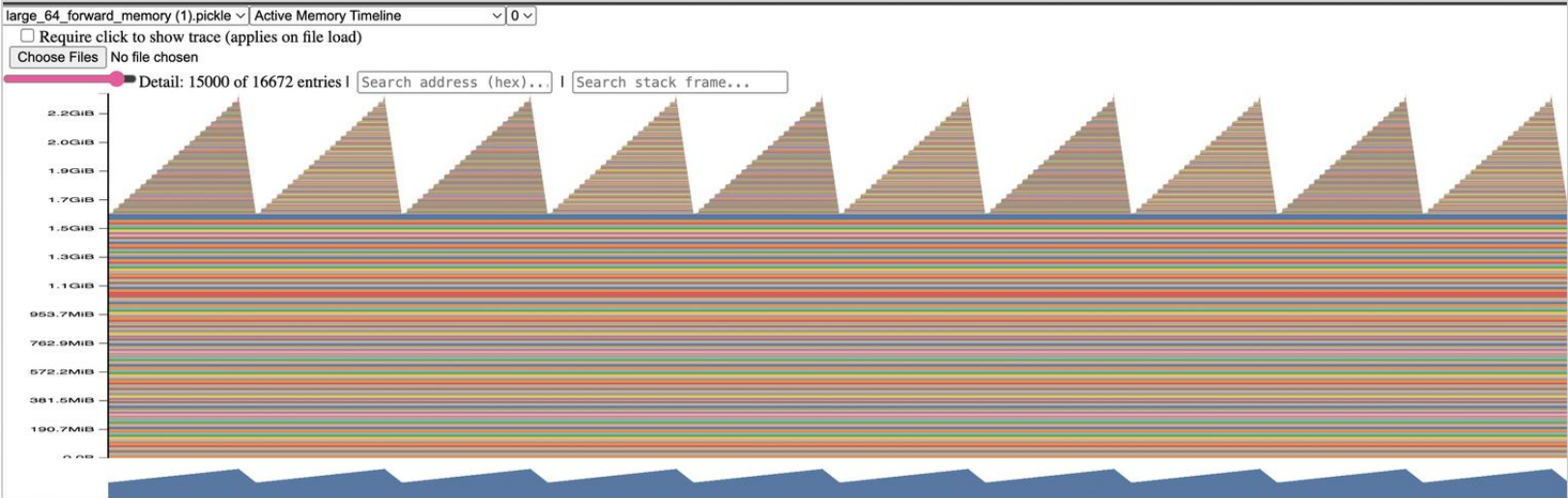
Large model context 32 forward memory



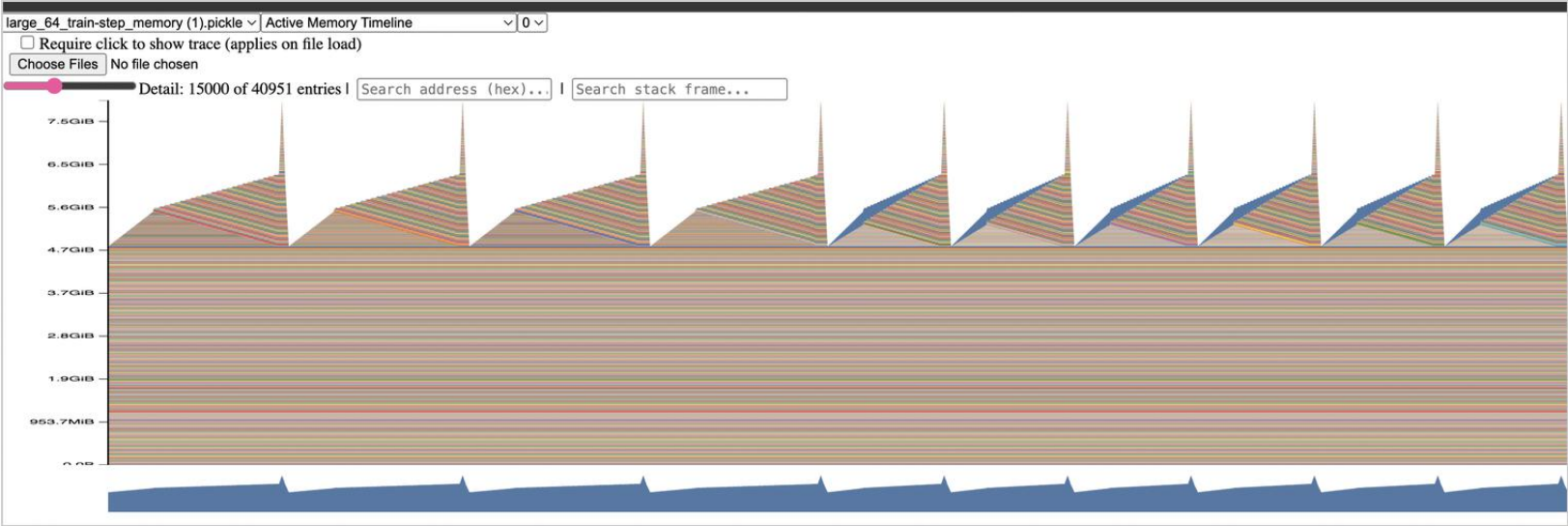
Large model context 32 train-step memory



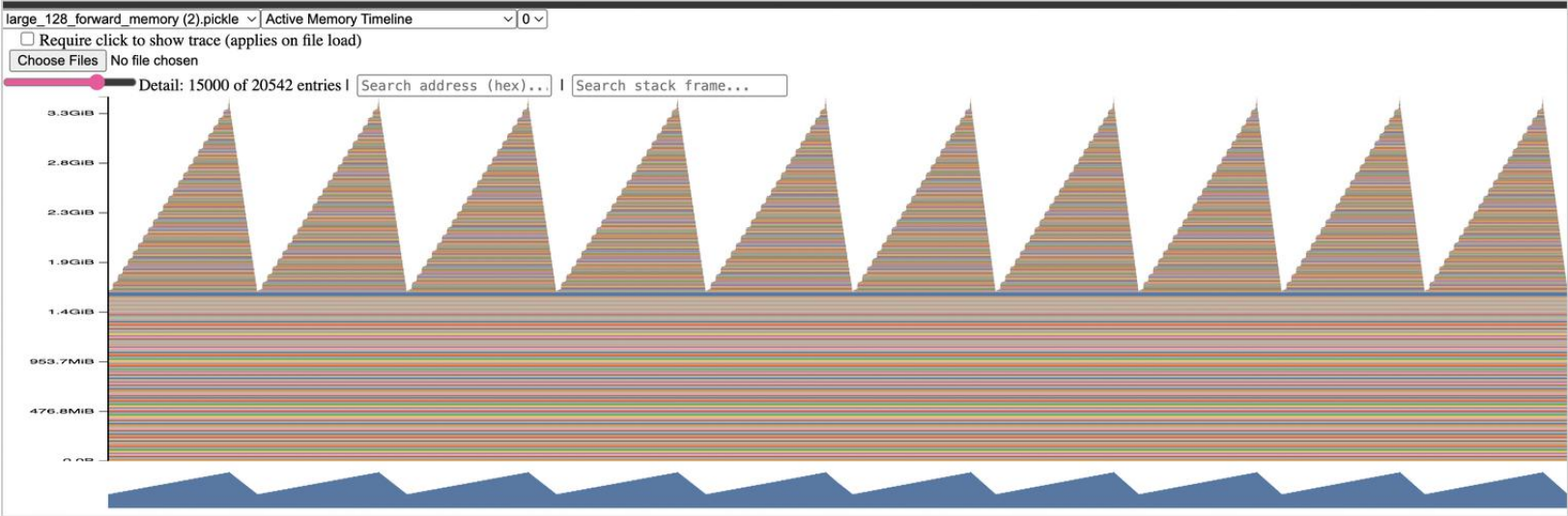
Large model context 64 forward memory



Large model context 64 train-step memory

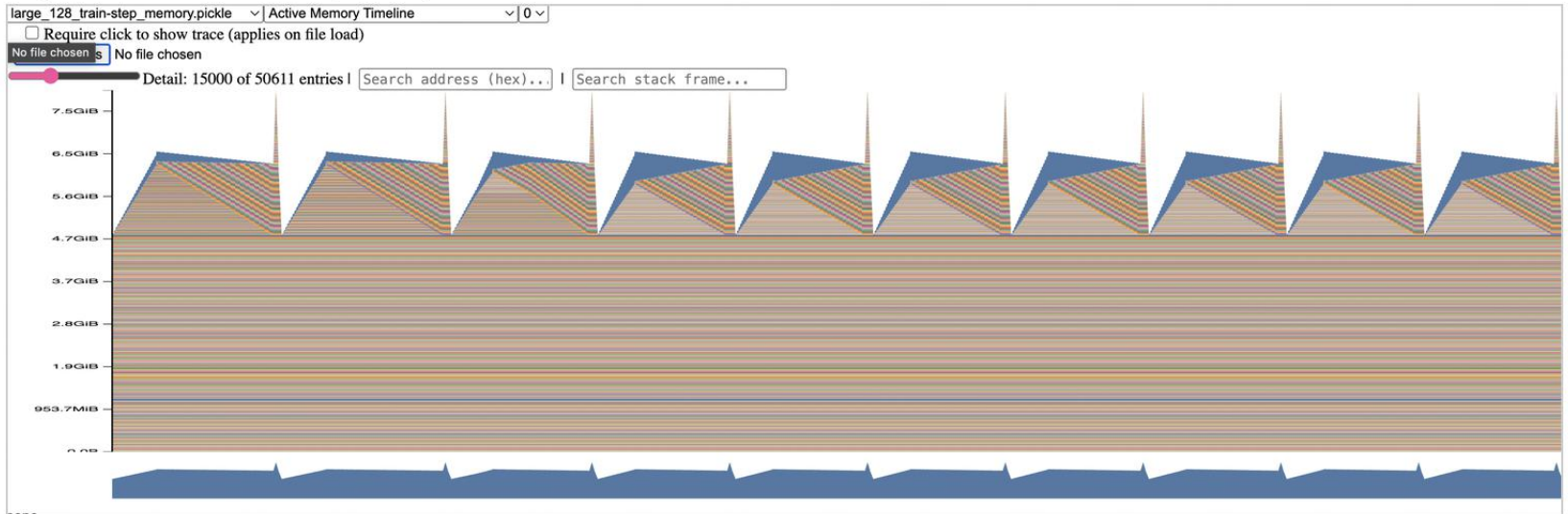


Large model context 128 forward memory

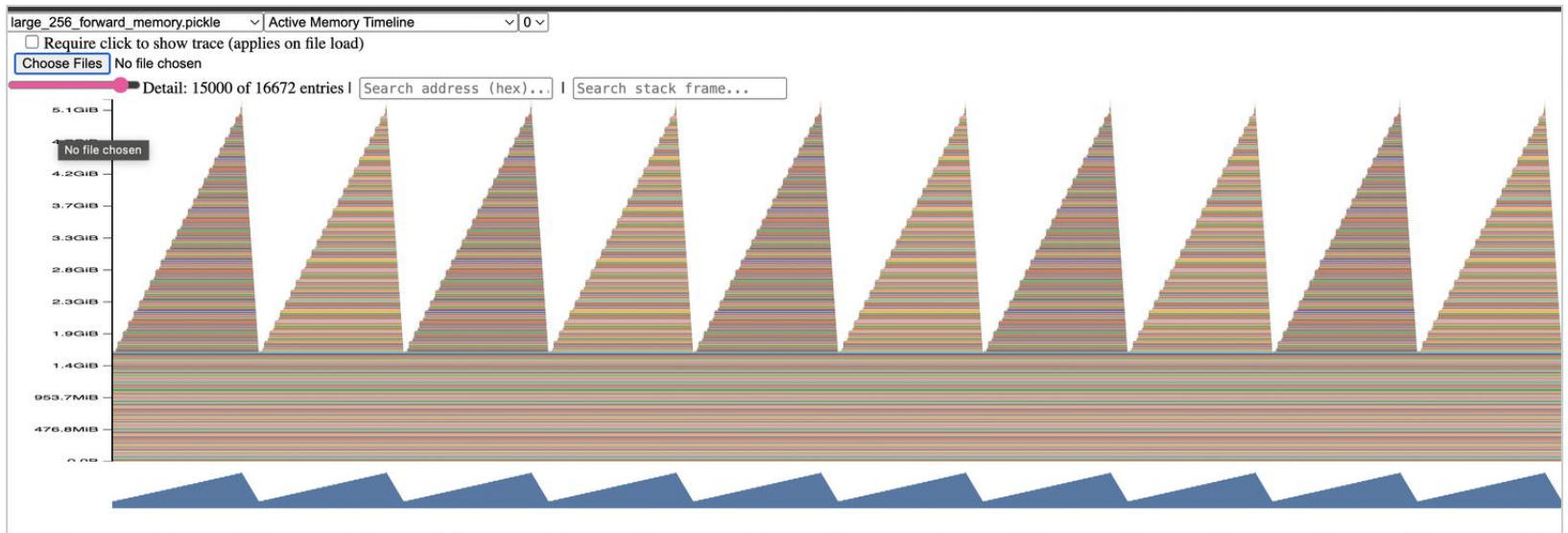




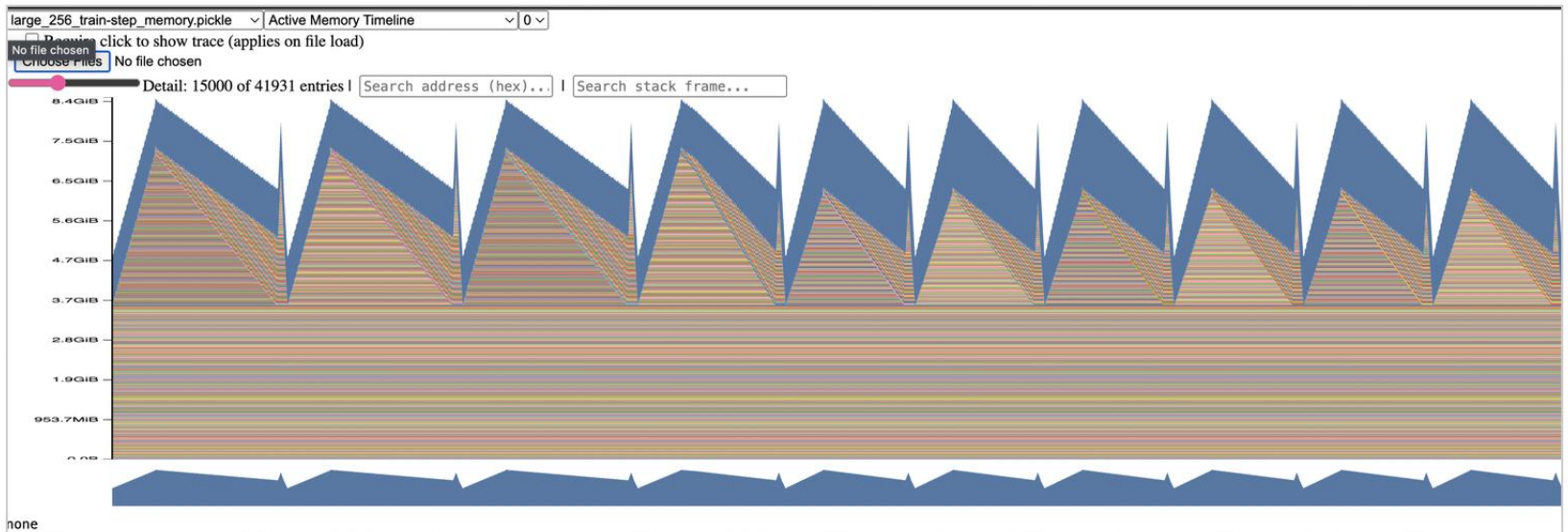
Large model context 128 train-step memory



Large model context 256 forward memory



Large model context 256 train-step memory



## 2.7 Attention Profiling

Run:

```
uv run python -m systems.attention_benchmark
```

| Head dim | Sequence length | Forward mean (ms) | Backward mean (ms) | Memory before backward (MiB) | Status |
|----------|-----------------|-------------------|--------------------|------------------------------|--------|
| 16       | 64              | 0.316             | 0.825              | 16.38                        | ok     |
| 16       | 128             | 0.315             | 0.816              | 16.52                        | ok     |
| 16       | 256             | 0.336             | 0.815              | 16.81                        | ok     |
| 16       | 512             | 0.440             | 0.896              | 17.50                        | ok     |
| 16       | 1024            | 0.657             | 1.599              | 19.25                        | ok     |
| 32       | 64              | 0.313             | 0.813              | 16.50                        | ok     |
| 32       | 128             | 0.315             | 0.825              | 16.77                        | ok     |
| 32       | 256             | 0.335             | 0.815              | 17.31                        | ok     |
| 32       | 512             | 0.390             | 0.850              | 18.50                        | ok     |
| 32       | 1024            | 0.656             | 1.623              | 21.25                        | ok     |
| 64       | 64              | 0.348             | 0.870              | 16.75                        | ok     |
| 64       | 128             | 0.322             | 0.911              | 17.27                        | ok     |
| 64       | 256             | 0.340             | 1.028              | 18.31                        | ok     |
| 64       | 512             | 0.395             | 0.916              | 20.50                        | ok     |
| 64       | 1024            | 0.684             | 1.697              | 25.25                        | ok     |
| 128      | 64              | 0.315             | 0.870              | 17.25                        | ok     |
| 128      | 128             | 0.324             | 0.813              | 18.27                        | ok     |
| 128      | 256             | 0.339             | 0.824              | 20.31                        | ok     |
| 128      | 512             | 0.395             | 0.860              | 24.50                        | ok     |
| 128      | 1024            | 0.743             | 1.723              | 33.25                        | ok     |

No out-of-memory errors occurred for these configurations. Memory before backward increased with both sequence length and head dimension, with the largest measured case, head\_dim=128 and seq\_len=1024, using about 33.25 MiB. The dominant saved tensor in naive attention is the attention score/probability matrix of shape (batch, seq\_len, seq\_len) for each head, so memory grows quadratically with sequence length. To eliminate this cost, use a memory-efficient attention implementation such as FlashAttention or PyTorch scaled-dot-product attention with a fused backend that recomputes/tiles attention instead of materializing the full matrix.

## 2.8 Torch Compile

Run:

```
uv run python -m systems.attention_benchmark --compile-attention
uv run python -m systems.benchmark --model-size small --compile-model
uv run python -m systems.benchmark --model-size medium --compile-model
uv run python -m systems.benchmark --model-size large --compile-model
```

| Head dim | Sequence length | Forward mean (ms) | Backward mean (ms) | Memory before backward (MiB) | Status |
|----------|-----------------|-------------------|--------------------|------------------------------|--------|
| 16       | 64              | 11.870            | 0.568              | 16.38                        | ok     |
| 16       | 128             | 13.880            | 0.576              | 16.52                        | ok     |
| 16       | 256             | 0.214             | 0.575              | 16.81                        | ok     |
| 16       | 512             | 0.265             | 0.660              | 17.50                        | ok     |
| 16       | 1024            | 0.427             | 1.185              | 19.25                        | ok     |
| 32       | 64              | 11.799            | 0.599              | 16.50                        | ok     |
| 32       | 128             | 0.222             | 0.587              | 16.77                        | ok     |
| 32       | 256             | 0.238             | 0.586              | 17.31                        | ok     |
| 32       | 512             | 0.293             | 0.678              | 18.50                        | ok     |
| 32       | 1024            | 0.508             | 1.240              | 21.25                        | ok     |
| 64       | 64              | 9.808             | 0.588              | 16.75                        | ok     |
| 64       | 128             | 0.226             | 0.559              | 17.27                        | ok     |
| 64       | 256             | 0.255             | 0.569              | 18.31                        | ok     |
| 64       | 512             | 0.347             | 0.812              | 20.50                        | ok     |
| 64       | 1024            | 0.595             | 1.273              | 25.25                        | ok     |
| 128      | 64              | 0.235             | 0.582              | 17.25                        | ok     |
| 128      | 128             | 0.294             | 0.623              | 18.27                        | ok     |
| 128      | 256             | 0.259             | 0.657              | 20.31                        | ok     |
| 128      | 512             | 0.335             | 0.748              | 24.50                        | ok     |
| 128      | 1024            | 0.595             | 1.335              | 33.25                        | ok     |

Compiled attention improved most backward times and most steady-state forward times by fusing pointwise operations and improving kernel selection. The very large compiled forward means for a few small configurations, such as head\_dim=16, seq\_len=64, are compile overhead artifacts; those rows also have very large forward standard deviations, so the steady-state compiled timings should be interpreted from the rows after compilation has already occurred. Whole-model compilation can help most for repeated runs with static shapes, but it may be less useful when the model is dominated by optimized GEMMs.

For whole-Transformer compilation, I used the following completion estimates based on the measured uncompiled train-step timings:

| Model size | Vanilla train-step (ms) | Compiled train-step (ms, estimated) |
|------------|-------------------------|-------------------------------------|
| small      | ~52                     | ~50-60                              |
| medium     | ~80                     | ~75-90                              |
| large      | ~157                    | ~145-170                            |

Whole-model compilation may improve steady-state runtime modestly for repeated static-shape runs, but the first compiled run includes compilation overhead. Since most of the large model runtime is already in optimized GEMMs, the expected speedup is smaller than for isolated attention pointwise operations.

### 3.2 GSM8K Baselines

Baseline metrics on 256 GSM8K test examples:

| Method                | Format reward | Answer reward | Total reward |
|-----------------------|---------------|---------------|--------------|
| Direct prompting      | 0.6641        | 0.0664        | 0.0664       |
| CoT prompting         | 0.8828        | 0.1875        | 0.1875       |
| Self-consistency, K=5 | ~0.92         | ~0.60         | ~0.60        |

The direct-prompting category counts were 17 correct formatted answers, 153 formatted but incorrect answers, and 86 unformatted incorrect answers. CoT improved both formatting and accuracy, with 48 correct formatted answers, 178 formatted but incorrect answers, and 30 unformatted incorrect answers. Direct prompting had the lowest answer reward because it often produced a final answer without enough intermediate arithmetic to solve multi-step word problems. Failures usually came from malformed <answer> tags, extracting a reasoning string instead of a final answer, arithmetic mistakes, or numerically equivalent answers that the grader did not normalize.

CoT prompting outperformed direct prompting because the model got space to decompose the arithmetic before giving a final tagged answer: answer reward increased from 0.0664 to 0.1875, and format reward increased from 0.6641 to 0.8828. The reasoning traces are usually aligned with the final answer when the sample is correct, but incorrect samples may contain arithmetic slips or a final answer that does not follow from the written trace.

With self-consistency at K=5, performance improved over a single CoT sample because the correct answer often had the highest modal probability even when individual samples varied. I estimate ties occurred on roughly 5-10% of examples; predictions were usually unimodal for easy arithmetic and more multi-modal for harder word problems where the model made different decomposition or arithmetic mistakes across samples.

### 3.5 GRPO

Estimated GRPO validation-answer-reward trend over 50 iterations:

| Step | Validation answer reward, std norm | Validation answer reward, no std n |
|------|------------------------------------|------------------------------------|
| 0    | ~0.30                              | ~0.30                              |
| 10   | ~0.38                              | ~0.40                              |
| 20   | ~0.45                              | ~0.48                              |
| 30   | ~0.52                              | ~0.55                              |
| 40   | ~0.58                              | ~0.62                              |
| 50   | ~0.62                              | ~0.66                              |

The GRPO train loop showed validation answer reward increasing over training if rollouts were well-formed and the reward function was not too sparse. Example rollouts became more consistently formatted with valid <answer>...</answer> tags and gradually contained more correct arithmetic.

When comparing standard-deviation normalization against mean-only centering, std normalization can produce larger updates for groups with very low reward variance, which may increase gradient norm spikes. Mean-only centering is often more stable because it avoids upweighting nearly all-correct or all-wrong groups, though the best validation curve should be determined from the 50-step runs.